

Health Data Sources I: Clinical Records and FHIR App Design

+ Week 02

Dr. Xuhai “Orson” Xu

AI @ VP&S Initiative



COLUMBIA

VAGELOS COLLEGE OF
PHYSICIANS AND SURGEONS

COLLABORATORY
AT COLUMBIA UNIVERSITY

DAP
Lab

Today's Outline

1. **Quick Recap from Week 1**
2. **Health Data with Different Modalities and Shapes**
3. **Clinical Records Preserve Workflow and Semantics**
4. **FHIR Exposes Data Through Bounded Interfaces**
5. **Design the FHIR Connection First**

Quick Recap from Week 1

Lecture and Hands-on Lab

This is a **hands-on course**:

- **Time:** Friday 8:30–11:30 AM ET
- **Location:** Mudd 825 -- *Watch out for an announcement for potential room change due to size*
- **Lecture** ~1.5-2 hrs: establish concepts and shared vocabulary. Mostly by Orson Xu and Eugene Wu.
- **Lab** ~1-1.5 hrs: apply them to a working artifact
- **Attendance:** submit at the end of the lab session for attendance, incomplete is fine.
- **By next Wednesday:** submit the same notebook after completing the take-home tasks and reflection

A large amount of content may not be covered extensively during the lecture, but learned during practice. So, get your hands dirty!

Course Materials

- Main Course Website: <https://personal-health-agent.github.io/>
 - Course syllabus, project description, weekly slides, most reading materials
- Github Links: <https://github.com/personal-health-agent/binf4070-2026>
 - Weekly slides
 - Python notebooks for Lab sessions
 - You will use Colab - More to elaborate in the lab session
- Course Works: <https://courseworks2.columbia.edu/courses/251854>
 - Assignments (Labs, quizzes, project), grades, announcements, and Ed Discussion forum
 - Some copyright-protected reading materials

Grading Policy

Component	Weight	Notes
Weekly labs	5% in class (10 x 0.5%) 15% take home (10 x 1.5%)	12–13 sessions; only 10 required.
Two quizzes	20% (2 x 10%)	Week 7 & 11 (on the class)
Proposal + presentation	15%	Week 9 (on the class)
Demo / showcase	15%	Week 14 (on the last class)
Final implementation	15%	Due: Week 15
Writeup + reflection	15%	Due: Week 15

Optional pre–Week 4 capstone project pitch: **+1% extra credit.**

Late policy: Three “grace days” per student/team across the term; no request needed. After that, 10% off per day unless arranged in advance.

Capstone Project

- You will form a team of **3–4**, ideally with diverse backgrounds
 - Our class has a balanced mix of health and CS students
- Each team will choose a specific target user group and question to address with a PHA system
- The system should use at least one of: (1) clinical/EHR, (2) everyday sensors, or (3) self-report source
 - Will introduce these three sources later in the lecture
 - Multiple data modalities are encouraged and preferred
- Build an LLM, RAG, or agent component with a usable interface
- The system can be any reasonable format: website, mobile app, desktop app, command-line tool, or something creative...

Capstone Project

Read more about the capstone project in the course syllabus:

personal-health-agent.github.io/project-details/

Start ideation and team formation early!

- Week 4: Project Pitch & Team Formation
- Week 9: Project Proposal with Low-Fidelity Presentation
- Week 14: Project Demo and Poster; Writeup Due Dec 16

Build with Your Own Data

- Each of you will have the option to get a **Fitbit Air** (wk 3).
 - If you get one, you agree that your final project will include Fitbit as part of the PHA system. Using your own data would be a plus.
 - You will need to sign an electronic form and upload your Student ID card to get the device.
 - You will need to return the device at the end of the course
 - Please try your best to protect the device!
- You are not required to get a device. **Synthetic data** provide a complete path for the labs and project
 - Getting a device does not affect your score; it is not part of the rubric



AI Use Policy

You are **encouraged** to use AI. One of the learning objectives is to learn how to prompt AI effectively.

Each of you will get a **pre-paid API key** (up to ~20M tokens, TBD, wk3). When used up, you are on your own. You are free to use your own API key or other AI tools.

When using AI, disclose the assistance, verify its outputs, and keep identifiable data protected.

You don't want to be replaced by AI in 2030. So I trust you that...

- You **won't skip the class** and imagine that AI can just do everything for you
 - Well, technically you *can*, but then why are you even taking the class? :)
- You **won't blindly throw the lab notebook to AI** and just copy the output
- You **will think creatively and critically** about the problem you want to solve - the humane part
- You **will verify the outputs from AI** and use them as a starting or ending point for your own work

Health Data with Different Modalities and Shapes

The same person can generate multi-modal data records with different clocks, purposes, and gaps.

One Question Meets Different Data Records

“Why has my fatigue worsened this month?”

- **Clinical/EHR** — a complete blood count (CBC) order and result after a visit
Contributes care context and medical findings
- **Everyday Sensors** — minute-level heart-rate estimates with gaps
Contributes repeated timing and change
- **Self-Report** — a nightly fatigue rating plus a short note
Contributes lived experience and context

No single row answers the question alone, but combined together provides a richer picture.

A Data Ecosystem that Joins Distinct Sources (from week-01)

A **personal health data ecosystem** is the set of records and data streams that can describe a person's health. Each source adds a partial view shaped by how and why it was produced.

Three representative modalities organize the evidence:

- **Clinical/Electronic Health Record (EHR)** — care delivery → diagnoses, labs, orders, and notes
- **Everyday Sensors** — Data (e.g., mobile, wearable, connected-device, and network traces) + algorithms → physiological estimates and behavioral features
- **Self-report** — a person's entry → symptoms and lived context

Choose and combine sources around the user's question. No single modality captures the whole picture of the user.

The Same Time Period with Three Rhythms

```
Clinical/EHR      | visit |--| lab |-----| visit |--| lab |--  
Everyday Sensors  ~~~~~ dense series ~~~ gap ~~~~~ dense series ~~~  
Self-Report       x    x  x    x    x x x    x  x          x
```

Clinical EHR: Sparse encounters; Everyday sensors: Dense but with gaps; Self-reports: Irregular

Data Shape Is More Than Format

A data source includes **who or what produced the data record, for what purpose, under which conditions, and after which transformations.**

A clinician-entered diagnosis, a vendor-derived (e.g., Fitbit) pulse estimate, and a patient-entered fatigue score are not interchangeable facts. Record Examples:

- **EHR: FHIR Observation** — a result or assertion (more in later slides about FHIR)
Contains effective[x], issued, status, performer, code, and unit
- **Wearable:** A timestamped sample or interval
Contains a measured or derived value, observation time, device, and method
- **Self-report:** A response event
Contains prompt and response time, item, scale, recall time window

Missingness Has Different Meanings

Recall from last week: *Missingness* as one of the important considerations

- No lab result may mean:
 - the test was not ordered, not completed, performed elsewhere, or not yet returned
- No wearable value may mean:
 - off-wrist time, poor signal, delayed sync, or unavailable access
- No diary entry may mean:
 - low burden, high burden, forgetfulness, or symptoms that made responding harder

Treat each absence as **unknown until more evidence arrives**.

Clinical Records Preserve Workflow and Semantics

An EHR (Electronic Health Record) records care as a packet of events, orders, results, codes, and narrative.

Before EHRs: The Paper Chart

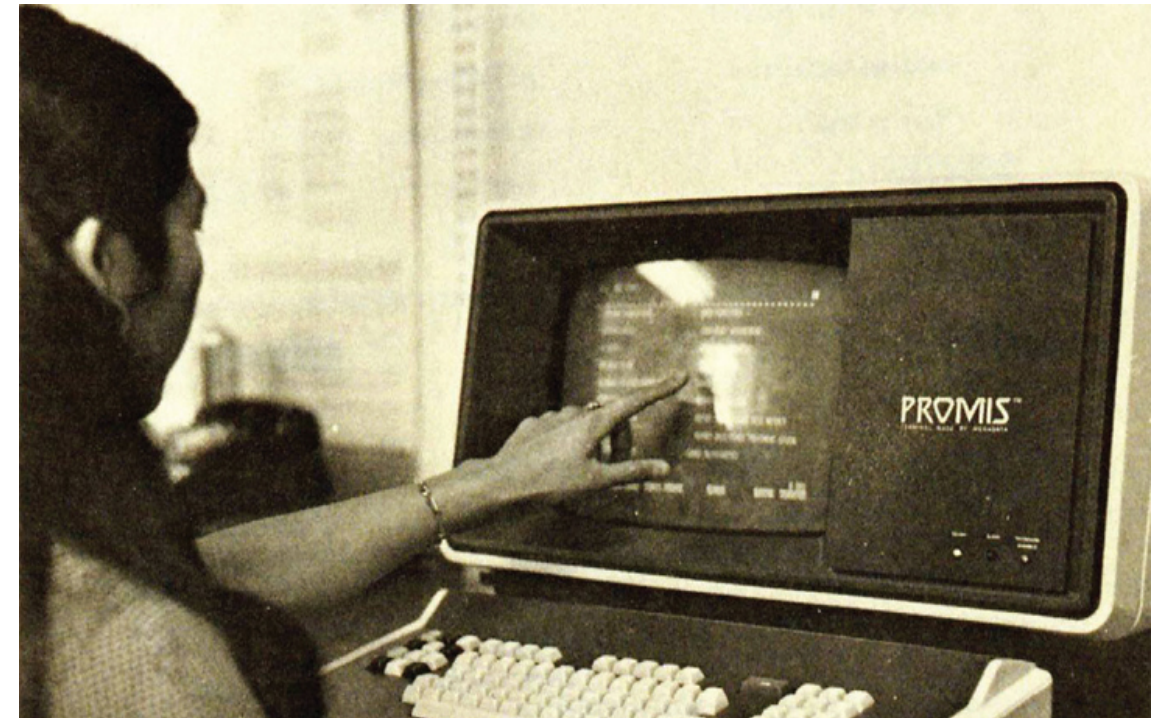
- **A visit generated several pages:** a clinician's note, orders or prescriptions, and later laboratory or radiology reports
- **Staff assembled one local dossier:** identifiers linked each page to the patient's folder
- **Using the record was physical work:** locate, file, read, copy, and transport the dossier

Clinical data existed, but retrieving or sharing it depended on moving paper.



Early-Stage EHRs

- **1968 — COSTAR:** Massachusetts General Hospital and Harvard Community Health Plan began implementing a computer-stored ambulatory record
- **Late 1960s–1970s:** HELP in Utah, Regenstrief in Indiana, and PROMIS in Vermont pursued different clinical workflows
- These systems tested structured records, orders, reminders, decision support, and retrieval—mostly within one institution



Adoption Came Through Standards and Policy

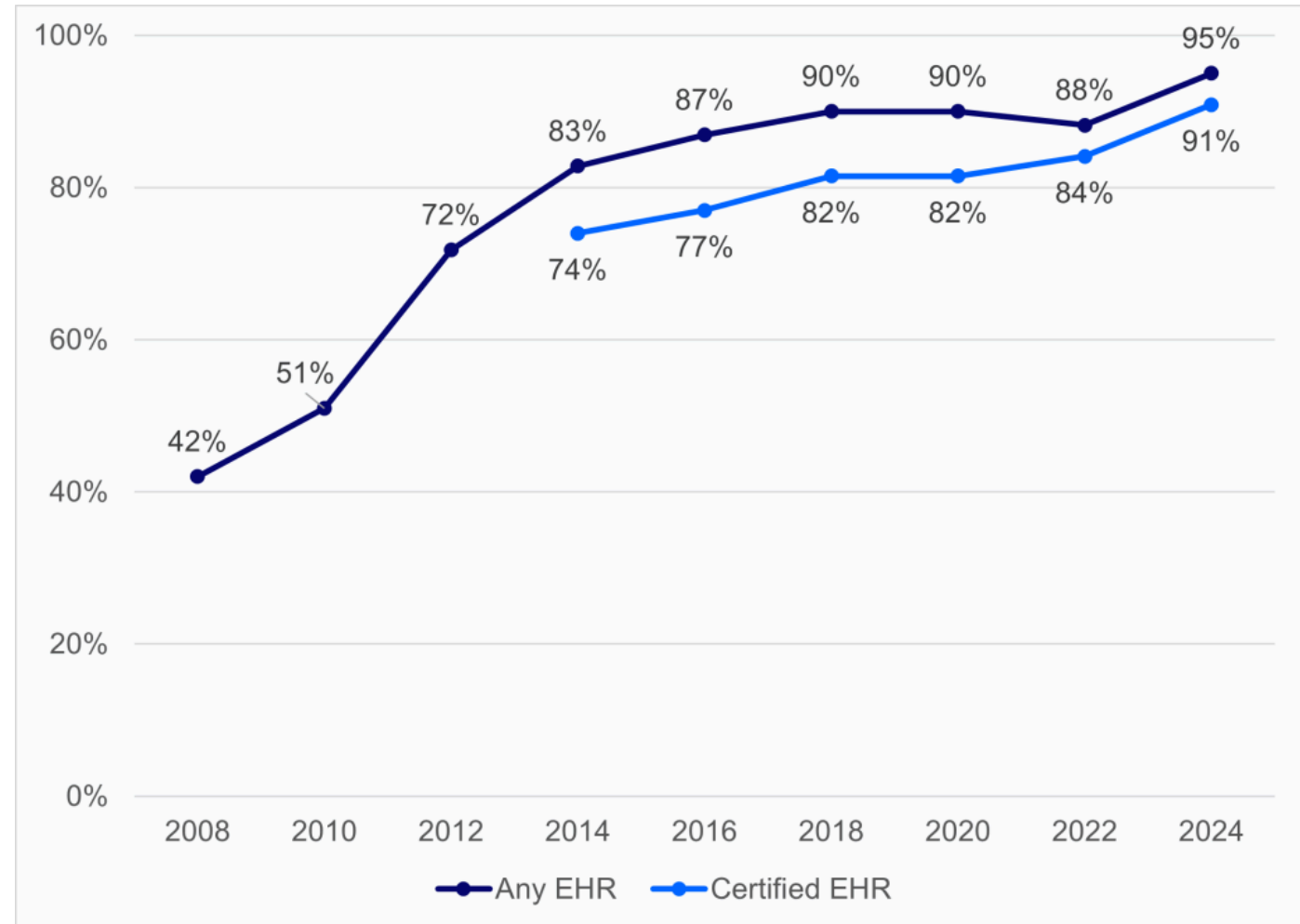
- **1987:** Health Level Seven (HL7) formed to standardize how clinical systems exchange information; “Level Seven” refers to the application layer of the Open Systems Interconnection (OSI) networking model
- **1990s–2000s:** local systems expanded while national and regional programs followed different paths across countries
- **2009 U.S.:** the Health Information Technology for Economic and Clinical Health (HITECH) Act funded adoption of certified EHR systems
- **Incentive payments began in 2011:** government health-insurance programs paid eligible clinicians and hospitals to adopt and use EHRs. Medicare required “meaningful use,” such as electronic prescribing. Medicaid also supported initial adoption
- **2008 → 2018 → Now:** U.S. hospitals with a Basic EHR rose from fewer than 1 in 10 to nearly all

Columbia's Integrated System Went Live in 1989

- **1987:** Columbia's College of Physicians and Surgeons and Presbyterian Hospital created the Center for Medical Informatics—the direct predecessor of today's Department of Biomedical Informatics (DBMI)
- **1989:** its integrated system went live with a clinical data repository, knowledge-based terminology, and automated decision support; later work added large-scale web clinical access and health information exchange
- **1994:** the center became a formal department and began graduate training, making it the second-oldest department of its kind in the United States (the oldest is at the University of Utah, back in 1972)
- Columbia paired an academic department with clinical system-building inside the medical center.

EHR Adoption In the Past Decades

- In 2024, 95% of office-based physicians reported using EHR
- 91% reported a certified EHR
- Three decades of digitization created an access path
- Adoption alone does not ensure cross-site completeness, aligned meaning, or usable exchange



What Data is Produced during A Modern Clinic Visit

A Record Packet:

Patient identity

|

Encounter -> Condition -> MedicationRequest -> Observation -> Procedure

|

Clinical note: symptoms, negation, uncertainty, and plan

Each artifact has its own author, status, timestamp, and reason for existing.

Coding Systems Create a Common Language

Clinical systems need stable identifiers because the same idea can be written as “high blood pressure,” “HTN,” or a local convention.

- **ICD-10-CM — International Classification of Diseases, 10th Revision (Clinical Modification):** diagnosis classification
- **Systematized Nomenclature of Medicine—Clinical Terms (SNOMED CT):** clinical concepts and their relationships
- **RxNorm — the U.S. National Library of Medicine's normalized drug vocabulary:** medication concepts
- **LOINC — Logical Observation Identifiers Names and Codes:** tests and observations
- **CPT — Current Procedural Terminology:** services and procedures

Their purposes differ even where their clinical coverage overlaps. The code system tells us how to interpret an identifier.

ICD-10-CM Classifies Diagnoses

ICD-10-CM is the U.S. diagnostic classification.

What does an ICD-10-CM code look like?

E11.9

- **E** — the E00–E89 chapter range covers endocrine, nutritional, and metabolic diseases
- **11** — completes category E11: type 2 diabetes mellitus
- **9** — subcategory: without complications

Examples: E11.9 = type 2 diabetes without complications · I10 = essential (primary) hypertension

SNOMED CT Identifies Clinical Concepts

SNOMED CT is a clinical terminology: identifiers, names, and relationships that represent clinical meaning. It covers findings, disorders, procedures, body structures, and more.

44054006

- **Fully specified name:** Diabetes mellitus type 2 (disorder)
- **U.S. preferred term:** Type 2 diabetes mellitus
- Both names describe **one concept** with the same numeric identifier

The digits identify the concept. Its descriptions and relationships to other concepts are stored separately; they cannot be read from the digits.

SNOMED CT and ICD-10-CM Complement Each Other

Serve entirely different purposes in healthcare: SNOMED CT is a detailed clinical terminology used for *patient care documentation*, while ICD-10 is a statistical classification system used primarily for *billing and epidemiology*.

One diabetes visit may need both a clinical problem entry and a diagnosis classification for reporting.

	SNOMED CT	ICD-10-CM
Main purpose	Represent clinical concepts and their relationships	Classify diagnoses for reporting and reimbursement
Example	44054006 · type 2 diabetes mellitus	E11.9 · type 2 diabetes mellitus without complications

44054006 alone does not establish “without complications.” Mapping to ICD-10-CM can require more clinical context and may produce more than one code.

RxNorm Normalizes Medication Concepts

The U.S. National Library of Medicine uses **RxNorm** to connect drug names from different vocabularies. **Rx** comes from the Latin word **recipe** (so it stands for a medical prescription or a treatment order).

ingredient + strength + dose form

- Fully specified generic name: metformin hydrochloride 500 mg oral tablet

An **RxNorm Concept Unique Identifier (RxCUI)** is the machine-readable code assigned to one normalized drug concept. It lets systems map different source names to the same medication meaning. The digits themselves do not encode ingredient, strength, or dose form. **Two examples:**

- 861007 = metformin hydrochloride 500 mg oral tablet
- 197361 = amlodipine 5 mg oral tablet

LOINC Identifies Tests and Observations

LOINC (Logical Observation Identifiers Names and Codes) indicates the question that a result answers. Note that the measured value (i.e., the result) is stored separately.

4548-4

- **4548** — an opaque serial identifier (3 to 8 digits); these digits do not describe the test
- **4** — a check digit calculated from 4548; it verifies that the complete code was copied plausibly

Base number	→ Mod-10 (Luhn)	→ Expected digit
4548	arithmetic check	4

- 4548-4 stands for Hemoglobin A1c/Hemoglobin.total in Blood

If someone mistypes the base number as 4549-4, recalculation expects a different final digit and flags the code. The check digit catches many copying errors; it carries no clinical meaning.

The Six-Part Name Is Separate from the Code

The identifier 4548-4 points to a LOINC database entry. That entry's **Fully-Specified Name** carries the clinical meaning:

Hemoglobin A1c/Hemoglobin.total:MFr:Pt:Bld:Qn:

Part	Value	Meaning
Component	Hemoglobin A1c/Hemoglobin.total	ratio being measured
Property	MFr	mass fraction
Time	Pt	one point in time
System	Bld	blood
Scale	Qn	quantitative value
Method	blank	no particular method specified

CPT Describes Services and Procedures

CPT (Current Procedural Terminology) is the American Medical Association's U.S. language for reporting professional services and procedures.

99213

- Category I codes are generally **five digits** and organized into broad service sections
- Do not parse 9 | 9 | 2 | 1 | 3: the individual digits do not carry independent clinical meanings
- The code set changes annually, so the **code-system version** matters

Two examples:

- 99213 = established-patient office or outpatient visit
- 93000 = routine 12-lead ECG with interpretation and report

Example: One Diabetes Visit Fans Out to Five Codes

Clinical problem -> SNOMED CT 44054006

Diagnosis class -> ICD-10-CM E11.9

Medication -> RxNorm 861007

HbA1c test -> LOINC 4548-4

Office service -> CPT 99213

Here, the diagnosis is separately documented as **without complications**. The first code alone does not justify the second.

Each system supplies an identifier for a particular purpose within the visit.

Code Identity Is system + code

A code becomes computable only when its **code system** travels with it. A display label helps a reader; source text can preserve local wording.

System + code	Identified concept
http://loinc.org 4548-4	Hemoglobin A1c in blood
http://snomed.info/sct 44054006	Type 2 diabetes mellitus

These are terminology identities, not links to a particular patient's record.

Clinical Notes Carry More Than Codes

“Reports worsening fatigue for three weeks, denies chest pain, will check TSH and CBC.”

- **Symptom:** worsening fatigue
- **Time:** three weeks
- **Negation:** denies chest pain
- **Plan:** check thyroid-stimulating hormone (TSH) and complete blood count (CBC)

One diagnosis field cannot preserve all four claims. Or content in a clinical note may be vague.

Clinical natural language processing (NLP) is an active research field. It focuses on extracts structure from notes while retaining context such as time, negation, and uncertainty.

The Complexity of Clinical Data

Clinical data require more context than these five coding systems and clinical notes can supply alone:

- A **vital sign** is entered inside a specific encounter workflow
- A **result** needs a unit, time, status, method, and performer
- A **problem-list** entry may be active, resolved, duplicated, or unreconciled
- A **note** can preserve uncertainty that a structured field cannot

The screenshot shows the OpenMRS Vitals screen for a patient named Betty Evans. The interface includes a top navigation bar with the OpenMRS logo, user information (admin), location (Outpatient Clinic), and a Logout button. Below the navigation bar, the patient's name (Betty Evans) and details (Female, 65 year(s), 25-Jul-1954) are displayed, along with a Patient ID (1005JK). The active visit is noted as 19-Sep-2019, 23:33:56, and the status is Outpatient. The Vitals section is active, showing a list of vital signs: Height (cm), Weight (kg), (Calculated) BMI, Temperature (C) (checked), Pulse (checked), Respiratory rate, Blood Pressure (selected), and Blood oxygen saturation. The Blood Pressure field is currently empty, with a '163' entered in the first input box. A 'Confirm' button is visible at the bottom of the Vitals section. The right side of the screen has 'Save Form' and 'Exit Form' buttons.

Why Meaning Still Diverges

- Base FHIR (EHR Exchange Standard) allows optional fields, extensions, and several terminology bindings
- Profiles and implementation guides narrow those choices, but jurisdictions, vendors, and institutions may adopt different ones
- Local data conventions (e.g., codes, units, status rules, mappings, and workflow definitions) still vary

Example: value: 7.0 is uninterpretable without the observation code and unit. Even with both, two sites may record different time windows or statuses.

A standard can align **syntax**. Shared profiles, terminologies, and data-quality rules are needed to align **meaning**.

Why EHR Data Exchange Still Struggles

- **Identity and discovery:** find the right endpoint and match the right patient
- **Access control:** authenticate the requester and authorize the permitted purpose and data
- **Governance:** reconcile consent directives, organizational policy, data-use agreements, and applicable laws
- **Operations:** maintain interfaces, compatible versions, mappings, and workable clinical processes

Usable exchange requires **technical connection, correct identity, permitted access, and operational trust.**

Break

FHIR: A Shared Exchange Standard

Why Healthcare Needs FHIR

Fast Healthcare Interoperability Resources (FHIR, pronounced “fire”) is HL7's standard for electronically exchanging health information.

Without a shared contract, every app-to-EHR connection needs custom answers to the same questions:

- What shape represents a patient, result, medication order, or claim?
- How are records identified, linked, requested, and returned?
- Which code systems and data formats are expected?

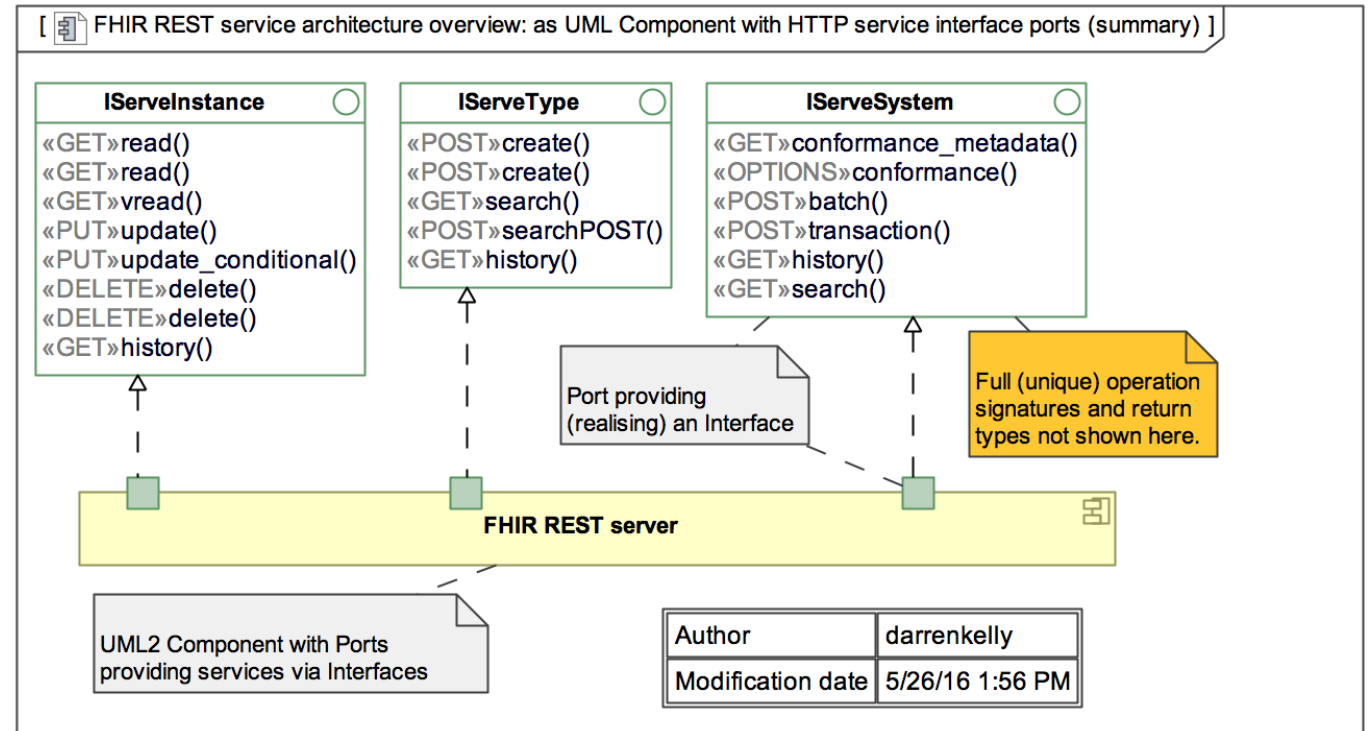
FHIR supplies reusable **data models and exchange rules**, so each connection does not begin with a new schema.

FHIR's Building Blocks

- **Resource:** a typed record such as Patient or Observation
- **Interaction:** a shared operation such as read, search, create, or update
- **Address:** [base]/[resource type]/[logical id]

GET /fhir/Patient/patient-001

The response is one Patient resource, usually represented as JSON or XML.



FHIR's Scope Has Boundaries

FHIR specifies:

- resource structures, identifiers, references, and terminology hooks
- testable exchange interactions and supported data representations
- conformance artifacts such as profiles and capability statements

FHIR leaves to implementations and policy:

- which resources, fields, profiles, and operations a server supports
- whether source data are complete, current, or clinically correct
- user interfaces, clinical workflows, identity matching, and access decisions

FHIR makes a shared exchange contract possible. It does not make every implementation or every record the same.

Four Properties Define a FHIR Resource

A FHIR resource is a structured unit that can be stored or exchanged. A typical resource:

1. **Has an identity** — normally a logical URL assembled from server base, resource type, and id
2. **Declares its type** — for example, Patient, Observation, or Condition
3. **Follows a defined structure** — named elements, data types, cardinalities, and constraints
4. **Has version identity** — the version changes when stored content changes

A resource is **one exchangeable unit**, not an entire patient chart.

Common Elements Form Two Layers

Every FHIR Resource can carry:

- `id` — logical id within its server and resource type
- `meta` — version, last update, profiles, security labels, and tags
- `implicitRules` and `language` — interpretation rules and base language

Most clinical and administrative types derive from `DomainResource`, adding:

- human-readable text, contained resources, extension, and `modifierExtension`
- type-specific elements such as `Patient.name` or `Observation.value[x]`

A **contained resource** is the identity exception: it has only a local `#id` inside its parent.

FHIR Resource Is the Base Abstraction

```
Resource (abstract base)
├─ DomainResource (abstract base)
│   ├─ Patient
│   ├─ Observation
│   ├─ Condition
│   └─ MedicationRequest
│       └─ ...other clinical and administrative types
├─ Bundle
├─ Binary
└─ Parameters
```

- Resource and DomainResource are abstract shared structure
- Patient, Observation, Condition, and MedicationRequest are concrete peer types

FHIR R4 Covers Clinical Work

R4 means Release 4, the fourth milestone release of FHIR. The permanent version used here is **4.0.1** (2019); the label tells us which resource definitions and API rules these examples follow.

- **People and organizations:** Patient, Practitioner, Organization, Encounter
- **Clinical care:** Condition, AllergyIntolerance, Procedure, CarePlan
- **Diagnostics:** Observation, DiagnosticReport, Specimen, ImagingStudy
- **Medications and immunization:** MedicationRequest, MedicationDispense, MedicationAdministration, Immunization

These representative resource types describe actors, care, results, and treatment across a clinical workflow.

FHIR R4 Also Supports the System

- **Conformance and terminology:** CapabilityStatement, StructureDefinition, CodeSystem, ValueSet
- **Security and documents:** Provenance, AuditEvent, Consent, Composition
- **Workflow and communication:** Task, Appointment, ServiceRequest, Communication
- **Finance, research, and reasoning:** Coverage, Claim, ResearchStudy, Library, Measure

FHIR R4 defines **145 resource types** across healthcare domains and supporting services. The official index lists them all: hl7.org/fhir/R4/resourcelist.html.

Four Examples Anchor This Lecture

Patient	Who is the record about?
Condition	What problem was asserted?
MedicationRequest	What medication was ordered?
Observation	What result or assertion was recorded?

These four resource types form a useful path through one clinic visit.

The same reading method applies elsewhere: identify the resource type, inspect its defined elements, follow its references, and preserve its provenance.

Example Visit

```
Patient/patient-001
  |-> Condition/cond-201
  |-> MedicationRequest/med-301
  |-> Observation/obs-401
```

A FHIR [Patient] Identifies the Person

Selected R4 fields from our visit example ("resourceType": "Patient" in JSON):

Element	Card.	Type	Example value
id	0..1	id	patient-001
identifier	0..*	Identifier	MRN-48291
name	0..*	HumanName	Maya Lee
birthDate	0..1	date	1985-04-12
managingOrganization	0..1	Reference	Organization/org-01

0..1 = optional single value; 0..* = optional repeating value. The complete resource has more fields.

A [Patient] Resource in JSON

```
{
  "resourceType": "Patient",
  "id": "patient-001",
  "identifier": [{
    "system": "https://hospital.example.org/mrn",
    "value": "MRN-48291"
  }],
  "name": [{
    "family": "Lee",
    "given": ["Maya"]
  }],
  "gender": "female",
  "birthDate": "1985-04-12"
}
```

Read from the outside in

- `resourceType` tells a parser which FHIR structure applies
- `id` identifies this resource on its server
- `identifier` carries a medical-record number and its assigning system
- `name` repeats because a Patient may have more than one name

A FHIR [Observation] Captures a Result

Selected fields behind Observation/obs-401:

Element	Card.	Type	Example value
status	1..1	code	final
code	1..1	CodeableConcept	LOINC 718-7 · hemoglobin
subject	0..1	Reference	Patient/patient-001
effective[x]	0..1	choice	2026-09-18T09:10:00-04:00
value[x]	0..1	choice	10.8 g/dL

Observation records evidence, not a diagnosis. A result may support a diagnosis; it does not establish one alone.

A FHIR [Condition] Records an Asserted Problem

Selected fields behind Condition/cond-201:

Element	Card.	Type	Example value
clinicalStatus	0..1	CodeableConcept	active
verificationStatus	0..1	CodeableConcept	confirmed
code	0..1	CodeableConcept	ICD-10-CM I10 · hypertension
subject	1..1	Reference	Patient/patient-001
onset [x]	0..1	choice	2022-06-15

Condition records an asserted problem. It stores the clinical claim and context, not the evidence that established it.

One [Medication], Three Different Events

1. **MedicationRequest** — **intended**

“Start metformin 500 mg twice daily.”

2. **MedicationDispense** — **supplied**

“Pharmacy handed over 60 tablets.”

3. **MedicationAdministration** — **given in care**

“Nurse administered 500 mg at 08:04.”

These resources can reference one another, but they are not interchangeable. An order can exist without a dispense; a dispense can exist without an observed administration.

References Connect Resources in Two Ways

No.1 Way:

```
Condition/cond-201                                Patient/patient-001
subject.reference -----^
Both resources have their own server identity.
```

No.2 Way:

```
Observation/obs-401
  device.reference -> #device-1
  contained[0] = Device#device-1
  #device-1 exists only inside Observation/obs-401.
```

A normal FHIR **reference** points to another resource with its own identity. A **contained resource** lives inside its parent and uses a local `#id`.

FHIR Read Versus Search

A FHIR **Bundle** is a resource that packages entry resources with metadata about why and how they were grouped.

```
GET /Patient/patient-001
```

→ one Patient resource

```
GET /Condition?patient=patient-001
```

→ a search-set Bundle, even with zero or one match

Search Parameters Encode the Question

A more realistic and complex search-set Bundle:

```
GET /Observation
  ?patient=patient-001
  &code=http://loinc.org|2345-7
  &date=ge2026-01-01
  &date=le2026-03-31
```

Retrieve this patient `patient-001`'s glucose Observations (LOINC 2345-7) recorded during the first quarter of 2026.

One Search Match Is Still a Bundle

```
{
  "resourceType": "Bundle",
  "type": "searchset",
  "total": 1,
  "entry": [{"resource": {"resourceType": "Observation",
                        "id": "obs-401"}}]
}
```

Parse `type` and `entry.resource` before flattening.

Bundle Type Determines Its Meaning

- **searchset** — matches plus search metadata and paging links
- **transaction** — operations the server must process atomically
- **history** — resource versions returned by a history interaction

Treating every Bundle as a bare list can silently change the operation. Check the bundle type carefully before any operation.

CodeableConcept (Code Systems!) Nested in Other Resources

Condition/cond-201	FHIR resource
└─ code : CodeableConcept	named element + data type
└─ coding[0] : Coding	nested data type
└─ system = http://snomed.info/sct	
└─ code = "38341003"	
└─ display =	
"Hypertensive disorder, systemic arterial (disorder)"	
└─ text = "High blood pressure"	source-facing wording

Condition and **Observation** are resources; **CodeableConcept** and **Coding** are nested data types. The outer `Condition.code` names a field; the inner `coding[].code` carries a terminology identifier.

Earlier Code Systems Fit Inside FHIR Fields

FHIR resource element	Terminology example	Example concept
Condition.code	ICD-10-CM	I10 · essential hypertension
Condition.code	SNOMED CT	38341003 · hypertensive disorder
Observation.code	LOINC	4548-4 · hemoglobin A1c
MedicationRequest.medication[x]	RxNorm	861007 · metformin 500 mg tablet
Procedure.code	CPT	93000 · routine 12-lead ECG

CodeableConcept is the **carrier**, not the vocabulary. The resource field and implementation profile determine which terminology is allowed.

A Safe Rendering Keeps Provenance

Too strong: “You have hypertension.”

Faithful to this record: “Your record lists an active Condition coded as hypertensive disorder, with onset recorded as June 15, 2022.”

The second sentence keeps the source, status, and recorded time visible. It leaves room for reconciliation and clinical interpretation.

SMART Connects Apps to FHIR

SMART stands for **Substitutable Medical Applications and Reusable Technologies**. SMART App Launch lets a third-party app launch in or alongside an EHR and obtain authorized access to a FHIR server.

Why FHIR alone is not enough

- FHIR defines resource shapes and API interactions
- It does not, by itself, tell an app how to discover OAuth endpoints, receive the current patient context, or request bounded permissions
- SMART adds server discovery, OAuth 2.0 authorization, launch context, and FHIR-aware scopes

SMART Launch Starts with Discovery

EHR / portal — iss Server address + launch reference → App
App — Request SMART configuration → FHIR server
App ← Authorization URL + token URL — FHIR server

- A standalone app starts with a known FHIR base URL
- iss ("issuer") gives the **FHIR base URL**, e.g. `https://ehr.example/fhir`
- Discovery: read `[base URL]/.well-known/smart-configuration`
- launch identifies this app launch and its EHR context.

PKCE Protects the Code Exchange

PKCE = Proof Key for Code Exchange. The app creates a fresh random secret for this access request.

- 1 App — Fingerprint of secret —————> Authorization server
- 2 App ← Code, if access is approved — Authorization server
- 3 App — Code + original secret —————> Authorization server
- 4 App ← Token, if the secret matches — Authorization server

The fingerprint is computed from the secret without revealing it. The server recomputes it to check the match. **A stolen code alone cannot obtain a token.**

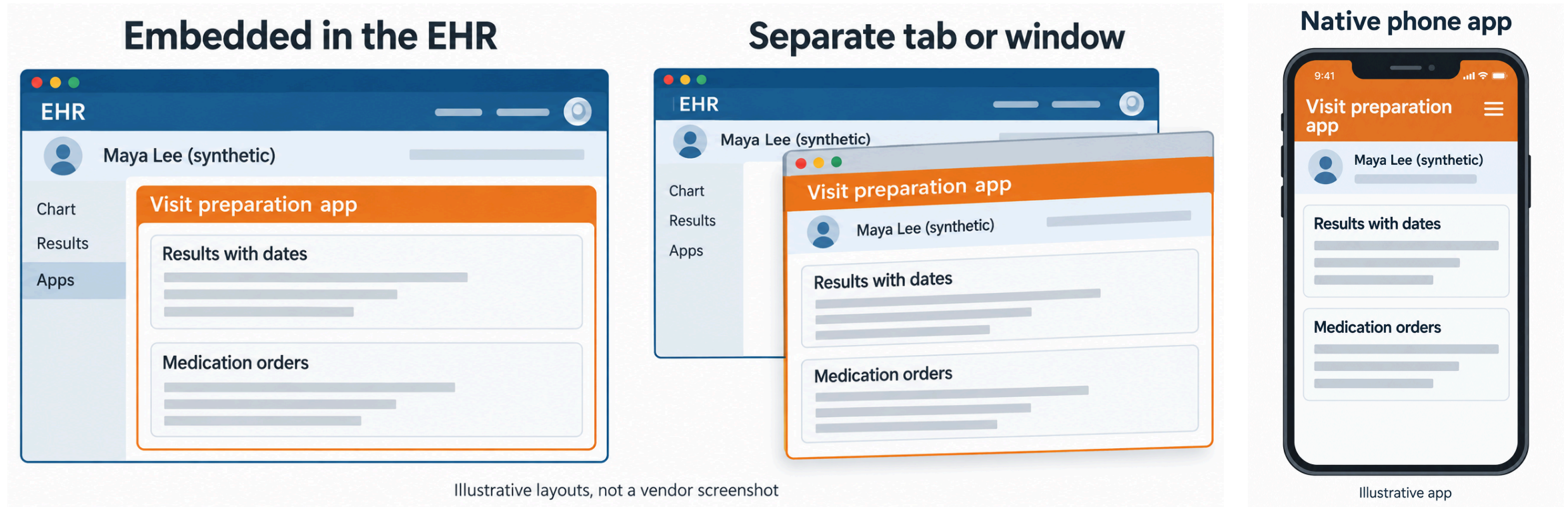
The App Requests Access to Results

Example: A clinician opens a lab-results app from a patient's EHR record.

App — Request access → Authorization server
Authorization server — Code via browser → App

- The **authorization server** decides access, checking login as needed, user approval when required, and hospital policy
- A **scope** names a permission, such as reading this patient's observations
- If approved, a short-lived **authorization code** returns through the browser. The app must exchange it for an access token

Where the SMART App Appears



- All three layouts can use the same SMART/FHIR access protocol.
- Opening the phone app directly is one example of a standalone launch.
- Phone sign-in uses a protected browser session, then returns to the app.

The Token Opens Access to Data

The app exchanges its authorization code at the **token endpoint**, the token-exchange URL found during discovery.

```
App — Code + PKCE secret —> Authorization server
App <— Access token — Authorization server
App — Token + lab query —> FHIR server
App <— Lab-result records — FHIR server
```

- The response includes the **granted scope** and patient context when available
- An **access token** is the app's credential for permitted data requests
- The **FHIR server** serves records and enforces access. The app sends authorization with its query

Design the FHIR Connection First

Move from a clinical question to a bounded data, authorization, and safety contract before choosing implementation details.

Start with One User Decision

Scenario: A care manager opens an EHR-launched app before a follow-up visit. The app should show recent blood-pressure readings beside active antihypertensive medication orders.

Before naming an endpoint, define:

1. **User and moment** — care manager, pre-visit review
2. **Decision supported** — what needs clarification during the visit?
3. **Minimum evidence** — readings, order status, relevant time, and source
4. **Unsafe shortcut** — do not equate an order with medication use or one high reading with a diagnosis

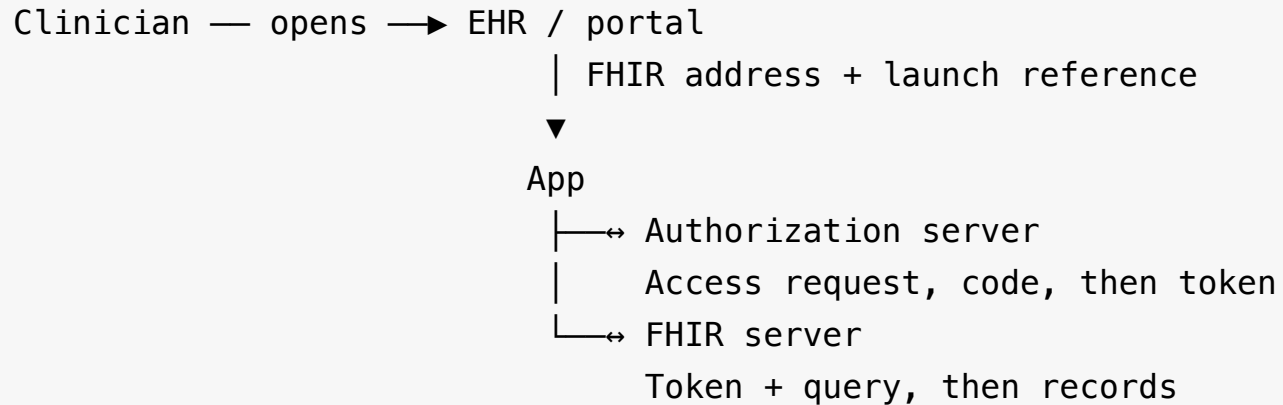
A useful FHIR app begins with a **workflow decision**, not a list of resources the server happens to expose.

Translate the Task into a Data Contract

Evidence needed	FHIR resource	Fields the app must preserve
Person in context	Patient or launch context	Patient identifier; never match by display name alone
Blood-pressure readings	Observation	LOINC code, status, effective time, units, systolic and diastolic components
Medication orders	MedicationRequest	RxNorm coding, status, intent, authored time
Visit context, if needed	Encounter reference	Encounter identifier, time, location, and organization

The contract states what the app needs and what each record can support. It does not promise that every server stores or exposes those fields in the same way.

Put Every Actor on the Boundary Map



- The **EHR/portal** is the screen where the clinician starts. It passes the launch context to the app
- The **authorization server** decides access. The **FHIR server** checks requests and supplies permitted records. One EHR system may provide both services
- The **app** displays the records with their sources and makes missing data visible

Decide what the app may read, retain, log, and display. Keep tokens and identifiable clinical content out of ordinary application logs.

Request Access by Job, Not Convenience

App job	Resource-specific access	Deliberately excluded
Review blood-pressure trend	Read/search Observation	Write access; unrelated resource types
Review medication orders	Read/search MedicationRequest	Administration or adherence claims
Receive the launched patient context	Launch context granted by the EHR	Searching across all patients

Example SMART scopes are `patient/Observation.rs` and `patient/MedicationRequest.rs`. The authorization server may grant less, and the app must honor the returned scope.

Design the Unhappy Paths First

Situation	Safe app behavior
Requested scope was not granted	Disable that feature and explain the access boundary
Search returned no matches	Say “no matching record returned,” not “normal”
Observation is preliminary	Show its status; do not present it as final
Unit or coding is unfamiliar	Preserve the source value and label; do not guess a conversion
Reference cannot be resolved	Keep the record visibly incomplete and avoid joining by name
Token expires	Reauthorize through the approved flow; never expose or paste the token

Testing Before Real Patient Use

1. **Review the design.** Agree on the user task, required records, and permissions before writing the connection
2. **Try local example files.** Test normal and missing records
3. **Connect to a public sandbox.** Use a test FHIR server with synthetic patients to check requests and responses
4. **Test with the partner organization.** Its separate test environment lets you check local data rules, access policies, and workflow
5. **Seek approval for real use.** Complete security and governance reviews, with monitoring and support in place before deployment

Each stage tests something the previous one could not. Fix problems with synthetic records before seeking access to real patient data.

Getting a Fitbit in Week 3

If you would like to borrow a Fitbit for the course:

- Complete and sign the electronic loan form, and upload your student ID through the form
- We will distribute the devices in **Week 3**
- Include Fitbit in your final PHA project is encouraged
- Take good care of the device and return it at the end of the course. **If you damage it, you may be responsible for repair or replacement costs**

Borrowing a device is optional. Synthetic data provide a complete path for the labs and project, with the same grading criteria.

Lab: Make a Readable Visit Summary

Turn one person's synthetic FHIR records into a card that keeps the result's value, unit, time, status, and source together.

In Class: Build and Check the Card

1. Read the linked `Patient`, `Observation`, and `MedicationRequest` records
2. Complete the helper, display the card, and add a reader note
3. Handle final, preliminary, and missing results; choose read and search access

Submit your notebook progress and visible outputs for attendance, even if some TODOs are incomplete.

Take Home: Add One Result

- Choose hemoglobin or glucose; reuse your card helper and formatter
- Add a short reader note; quote one result-related line and name its supporting fields
- State one limit of the chosen result, naming its source id
- Complete the learning and AI-use reflection

Submit the same completed notebook with visible outputs by the following Wednesday.

Thanks for the Attention
Q&A